

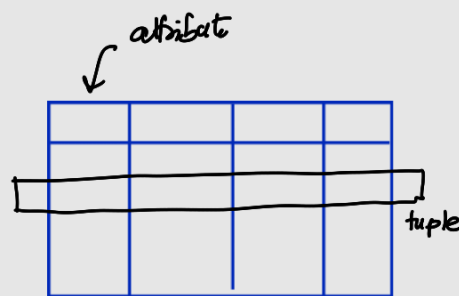
DBMS Module 2

• Relational Data Model

Primary data model, used for data storage and processing.
Simple model that has properties and capabilities to process data with storage efficiency. Represents data as collection of tables

The concepts used are:

1. Table: rows and columns → attributes
↓
represent records



Stores relation among entities.

2. Tuple: Single row of a table, contains single record for a relation.

3. Relation instance: Finite set of tuples that represent relation

instance. It don't have duplicates.

4. Relation schema: Describes relation name, attributes & their names

eg: STUDENT (Name, R-no, Age, Phone, Grade)

5. Relation key: Attributes that can identify the row in table uniquely.

6. Attribute domain: Predefined value scope.

eg: student - age: Possible age of students in college.

7. Degree / arity: No. of attributes in a relation schema.

8. Cardinality: Total no. of tuple present in a relation

STUDENT			
R-no	Name	Age	Mark
1	Anna	21	100
2	Benny	22	120
3	Cathy	20	140

Diagram annotations: An arrow labeled 'relation name' points to 'STUDENT'. An arrow labeled 'attribute' points to the header row. An arrow labeled 'Tuples' points to the three data rows.

- Relational Database Constraints

Conditions that must hold for it to be a valid relation. This is to hold reliable and accurate data.

Based on databases:

1. Inherent model-based: Inherent in data model. No need to mention explicit already exists

eg: Duplicate tuples.

2. Schema based: Defined directly in the schemas of the data model.

3. Application based: Must be expressed and enforced by application programs.

→ Schema based constraints:

1. Key / Entity constraints:

Must be at least one minimal subset of attributes in a relation.

More than 1 minimal subset ^{key} → candidate key.

→ No NULL value for key attributes

→ No 2 tuples can have identical values for key attributes

Properties: Unique Identification, Non-redundancy

eg: In Students table, R_no is key, not null and unique.

2. Domain Constraints

Every attribute is bound to have a specific range of values. It must be an atomic value, performs data type checks.

eg: Age: 21, 22, AB
✓ ✓ ✗

3. Referential Integrity Constraints

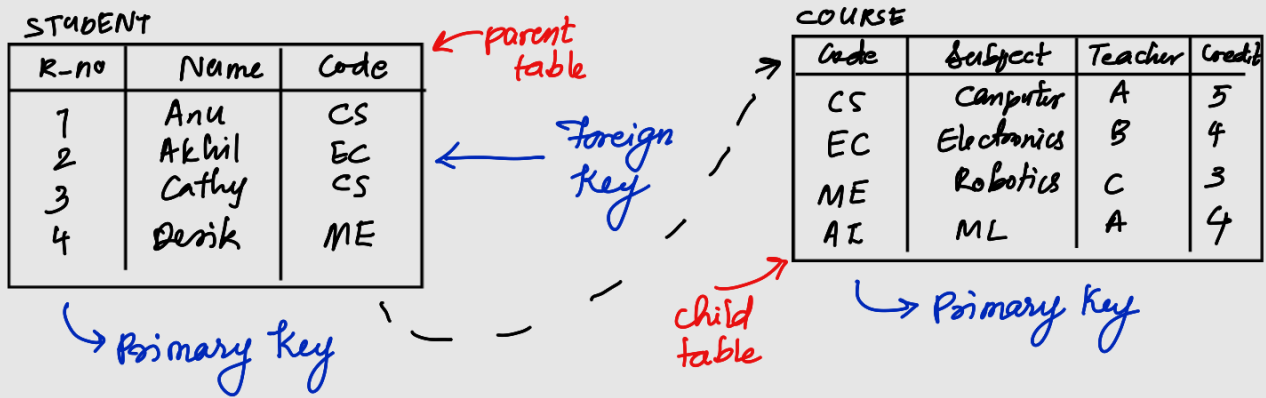
B/w 2 relations (tables).

Foreign key: attribute that can be referred in another relation.

It states that if a relation refers to a key attribute of a different or same relation, then that key element must exist.

1) Primary Key

2) Foreign Key



Foreign key must satisfy the following:

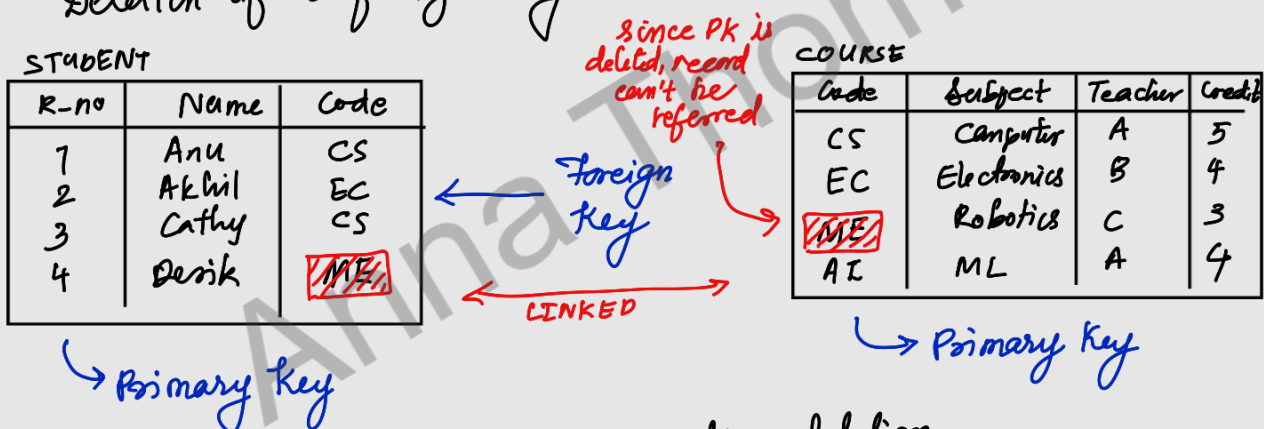
1. Same domain
2. Value of FK in a tuple either occurs as a value of PK.

$$t_p[FK] = t_c[PK] \text{ or is null.}$$

eg: For code, we can't have EI since it is not in table 2.

Primary objective: prevent orphans (records that can't relate to parent table) and enforce referential integrity

Deletion of a foreign key:



can lead to domino or cascading deletion.

Relational Algebra

also known as Procedural Query language / Formal Query lang.

Used to access data from database using mathematical expressions.

mention $\longrightarrow$ what use operators to perform queries.

$\longrightarrow$ how

BASIC OPERATORS:

- Projection (π)
- Union ($\cup$)

- Selection (σ)
- Rename (ρ)

(Cartesian Prod)

- Cross Product ($\times$)
- Set Difference ($-$)

1. Projection (π)

Projects columns that satisfy given predicate

Notation: $\pi_{A_1, A_2, \dots, A_n}(r)$
 (Attributes) (relation)

STUDENT

R-no	Name	Code	Marks
1	Anu	CS	45
2	Akhil	EC	30
3	Cathy	CS	50
4	Derik	ME	21

$\pi_{R-no, Name}(Student)$

O/P:

R-no	Name
1	Anu
2	Akhil
3	Cathy
4	Derik

2. Select (σ)

Selects tuples that satisfy the given predicate from a relation

Notation: $\sigma_p(r)$
 (selection) (relation)

STUDENT

R-no	Name	Code	Marks
1	Anu	CS	45
2	Akhil	EC	30
3	Cathy	CS	50
4	Derik	ME	21

Q: Select tuples whose code is CS
 $\sigma_{Code = "CS"}(Student)$

Q: Select tuples whose marks > 25
 $\sigma_{Marks > 25}(Student)$

Q: Find students that has marks > 25
 $\pi_{Name}(\sigma_{Marks > 25}(Student))$

3. Union ($\cup$)

Binary union b/w two relations and is defined as

$$r \cup s = \{t \mid t \in r \text{ or } t \in s\}$$

Conditions:

→ r & s must have same no. of attributes

→ Attribute domains must be compatible

→ Duplicate tuples are automatically eliminated

eg: $\pi_{author}(Books) \cup \pi_{author}(Articles)$

BOOKS

1	2
Author	Num
1	Amit
2	Aby
3	Asha

ARTICLES

1	2
Author	Num
1	Aby
2	Anu

4. Set Difference ($-$)

Tuples present in one but not in second $[r-s]$

eg: $\pi_{author}(Books) - \pi_{author}(Articles)$

5. Cross Product ($\times$)

Combines information of two different relations into one.

Notation: $r \times s$

$$r \times s = \{qt \mid q \in r \text{ and } t \in s\}$$

eg: $\pi_{author} = 'Aby' (Books \times Articles)$

Books $\times$
Articles

Author (B)	Num (B)	Author (A)	Num (A)
Amit	B1	Aby	A1
Amit	B1	Anu	A2
Aby	B2	Aby	A1
Aby	B2	Anu	A2
Asha	B3	Aby	A1
Asha	B3	Anu	A2

No of columns:
 $m+n$

No of rows:
 $x \times y$

6. Rename (ρ)

Rename o/p relation

Notation: $\rho_{name=x}(E)$
↖ name = x
↖ expression

DERIVED OPERATORS:

- Join ($\bowtie$)
- Intersect ($\cap$)
- Division ($/$)

1. Join ($\bowtie$)

Equijoin $\rightarrow$ Based on equality of column values, the join happens.

Natural Join $\rightarrow$ Equijoin but redundant columns removed.

STUDENT

R-no	Name	Code
1	Anu	CS
2	Akhil	EC
3	Cathy	CS
4	Desik	ME

COURSE

Code	Subject
CS	Computer
EC	Electronics
ME	Robotics
AI	ML

Equijoin:

R-no	Name	Code	Code	Subject
1	Anu	CS	CS	Computer
2	Akhil	EC	EC	Electronics
3	Cathy	CS	CS	Computer
4	Desik	ME	ME	Robotics

Natural Join:

R-no	Name	Code	Subject
1	Anu	CS	Computer
2	Akhil	EC	Electronics
3	Cathy	CS	Computer
4	Desik	ME	Robotics

2. Intersect ($\cap$)

$$(X \cap Y) = X - (X - Y)$$

common tuples to both X and Y

- Relational Calculus

Quantifiers
Operators
Attributes/Constants
Connectives

Non procedural query language (no need to give the procedures)

Notation: $\{t \mid P(t)\}$

t → tuple
 $P(t)$ → predicate

Existential quantifiers: $\exists$ or $\forall$

eg: $\exists r$ and $t \leftarrow$ free variable

Tuple variables can be compared to itself / constant

We have formulas P_1 and P_2

$\neg P_1$, (P_1) , $P_1 \wedge P_2$, $P_1 \vee P_2$, $P_1 \Rightarrow P_2$

If $P_1(s)$ then $\exists s \in r(P_1(s))$ and $\forall s \in r(P_1(s))$
 s → free tuple variable
 r → relation

Example:

- Find the R-no, name, dept, marks for the students whose mark > 20

$\{t \mid t \in \text{student} \wedge t[\text{mark}] > 20\}$

- Find R-no of student whose mark > 20

$\{t \mid \exists s \in$

$\text{student} (t[\text{R-no}] = s[\text{R-no}] \wedge s[\text{mark}] > 20)$

- Find student names from CS dept and mark > 20

$\{t \mid \exists s \in \text{student} (t[\text{name}] = s[\text{name}] \wedge s[\text{dept}] = \text{"CS"} \wedge s[\text{mark}] > 20)$

- Structural Query Language (SQL) → query structured data

Declarative language and it is domain specific. used to communicate with database

① DDL

② DML

③ DCL

④ TCL

3. Division ($/$)

used for queries involving 'all' or 'every'

STUDENT

R-no	Name	Dept	Mark
1	Angali	CS	25
2	Benedict	EC	27
3	Clara	AD	31
4	Diana	EC	20
5	Ellen	AD	15
6	Fiona	CS	25

usage of keys and constraints, operators (like, in, not in), clauses (order by, group by)

a) Data Definition Language

CREATE TABLE table-name;

(creates database & its objects)

ALTER TABLE table-name ADD COLUMN R-no INT;

DROP TABLE table-name;

(deletes obj from database: table removed)

TRUNCATE TABLE table-name;

(removes all spaces allocated for records)

RENAME TABLE table-nameold To table-name new;

b) Data Manipulation Language

SELECT Col1, Col2 FROM table-name;

INSERT INTO table-name (Col1, Col2) VALUES (val1, val2);

(Insert data)

UPDATE table-name SET col1 = val1 WHERE condition;

DELETE FROM table-name WHERE condition;

(delete records from database table)

Exercise:

A company wants to maintain a database for employees.

1. Create table EMPLOYEES with following fields:

- EmpID: Int, Primary key
- Name: Varchar (50), Not null
- Dept: Varchar (30)
- Salary: Decimal (10,2)

A: CREATE TABLE Employees (EmpID INT PRIMARY KEY,
Name VARCHAR (50) NOT NULL,
Department VARCHAR (30),
Salary DECIMAL (10,2));

2. Insert the records into the table

EmpID	Name	Dept	Salary
101	Amal	HR	15000
102	Neha	IT	25000
103	Rajat	Finance	20000

A: INSERT INTO Employees (EmpID, Name, Department, Salary)
VALUES (101, 'Amal', 'HR', 15000), (102, 'Neha', 'IT', 25000),
(103, 'Rajat', 'Finance', 20000);

3. Increase salary of IT employees by 10%.

UPDATE Employees SET Salary = Salary + (Salary * 0.1) WHERE
Department = 'IT';

4. Delete employee with ID = 101

DELETE FROM Employees WHERE EmpID = 101 ;

5. Display all employees whose salary is greater than 20000

SELECT * FROM EMPLOYEES WHERE Salary > 20000

- Assertions

used to specify integrity constraints that can't be specified by built-in relational model constraints.

DBMS enforces the assertion on the database and the constraints specified in assertions must not be violated.

Syntax: CREATE ASSERTION assertion-name CHECK condition ;

eg: Price of a notebook must not be greater than price of textbook

CREATE ASSERTION price-check CHECK
(NOT EXISTS (SELECT * FROM BOOK WHERE Catg = 'Notebook' AND
Price > (SELECT Min(Price) FROM BOOK WHERE Catg = 'Textbook'))));

Assertion is violated whenever the result of query inside NOT EXISTS is not empty.

- Triggers

To specify the type of **action** to be taken when certain **events** occur and when certain **conditions** are satisfied.
eg: Students marks reaching below 20 then parents will be notified.
Condition is used to monitor the database and then if the condition is satisfied it will trigger a specific stored procedure.

Uses: 1. Enforcing referential integrity

2. Security Authorization

3. Prevent Invalid transactions.

SYNTAX: CREATE TRIGGER trigger-name
[BEFORE/AFTER] [INSERT|UPDATE|DELETE] ON table-name
[for each row | for each column]
[TRIGGER BODY] $\longrightarrow$ procedure with no args.

eg: CREATE TRIGGER update_timestamp BEFORE UPDATE ON
users FOR EACH ROW

BEGIN

SET NEW.updated_at = CURRENT_TIMESTAMP;

END;

- Views

Virtual table that is formed from the result set of a stored query.
It will be based on a base table, once changed will reflect on the view and vice versa.

Types:

1. Simple view (from single table)

2. Complex view (from multiple tables)

We can create, delete, update view in DBMS.

Advantages:

1. To restrict data access

2. Data independence

3. Make complex queries easy

4. Present diff views of same data.

SYNTAX: CREATE VIEW view_name AS SELECT Col1, Col2 from Table1 WHERE condition;

(Multiple tables)

CREATE VIEW view_name AS SELECT Col1, Col2 from Table1, Table 2 WHERE condition;

Table1.col1 = Table2.col1

eg: CREATE VIEW Student View AS SELECT R.no, Name FROM STUDENT WHERE Dept = 'CS';

CREATE VIEW Student View AS SELECT Student. R.no, Student. Name, Course. Name FROM Student, Course where Student. Course Name = Course. Name;

- Relational Database Design Using ER to Relational Mapping

Mapping of

1. Regular Entity Types

Entity

2. Weak Entity types

3. Binary 1:1 Relation Types

Relationship

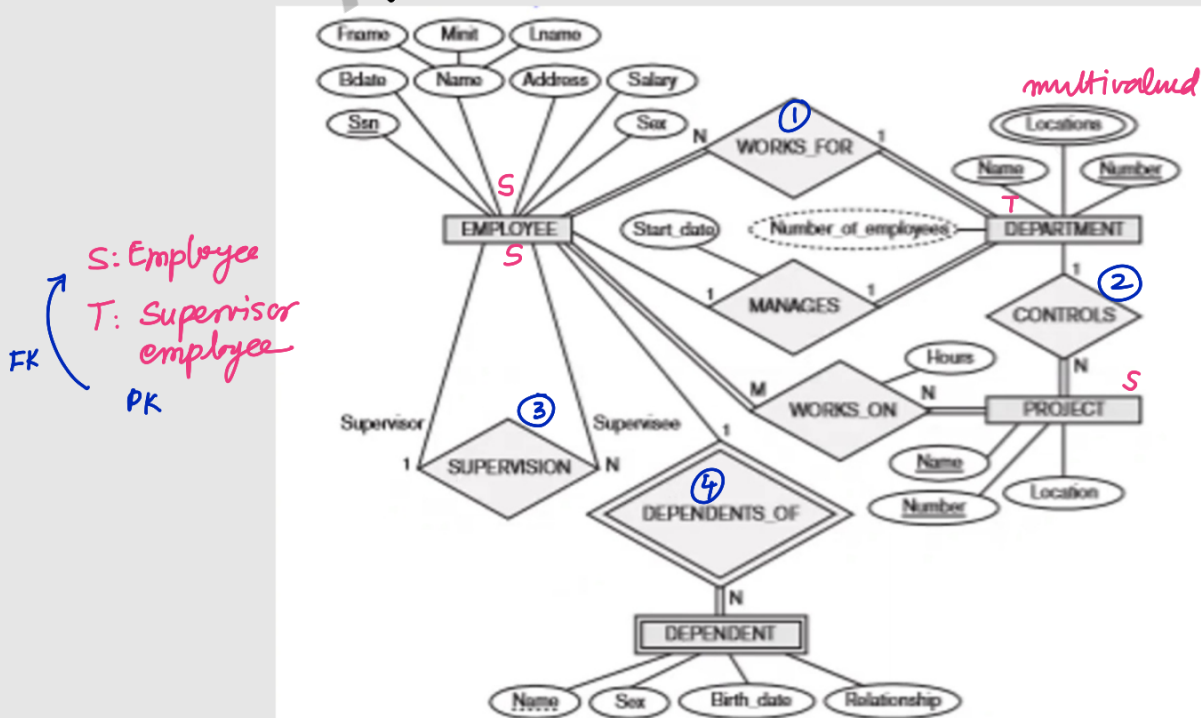
4. Binary 1: N Relationship Types

5. Binary M: N Relationship Types

Attributes

6. Multivalued attributes

7. N-ary relationship types



1) Mapping of Regular Entity Types

- For each strong entity type in ER diagram, create relation R that includes all simple attributes of E.
- choose one key attributes of E as primary key.
- If E is composite, set of simple attributes of E forms primary key of R.

EMPLOYEE

Fname	Mname	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary
-------	-------	-------	------------	-------	---------	-----	--------



DEPARTMENT

Dname	<u>Dnumber</u>
-------	----------------

PROJECT

Pname	<u>Pnumber</u>	Plocation
-------	----------------	-----------

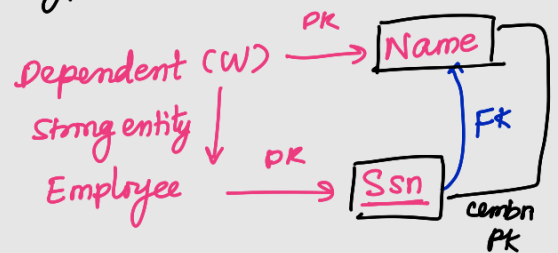
2) Mapping of Weak Entity Types

- Create relation R and include all simple attributes of the entity as attributes of R
- Include owner entity's primary key as R's foreign key.
- Primary key of R is the combination of the primary key of the owner and partial key of the weak entity type W.

DEPENDENT

<u>ESSn</u>	<u>Dep_name</u>	Sex	Bdate	Relationship
-------------	-----------------	-----	-------	--------------

foreign key



3) Mapping of Binary 1:1 Relationship Types

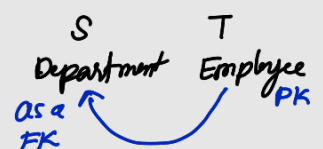
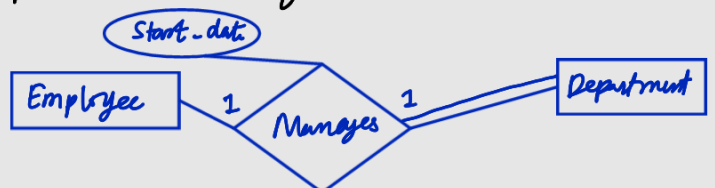
- Identify relations S and T that correspond to the entity types participating in R.
- choose one of the relations (S) & include a foreign key in S that is primary key of T. Better to choose total participation relation for S.

DEPARTMENT

Dname	<u>Dnumber</u>
-------	----------------

primary key of T

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------



4) Mapping of Binary 1:N Relationship Types

- Identify S representing the entity type at N-side of the relationship type.
- Include as foreign key in S, the primary key of T.
- Include simple attributes of 1:N relation type as attributes of S.

S
employee dept

EMPLOYEE

Fname	Mname	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary
-------	-------	-------	------------	-------	---------	-----	--------

EMPLOYEE

Fname	Mname	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Supr-Ssn	Dnum
-------	-------	-------	------------	-------	---------	-----	--------	----------	------

Dept PK

supervisor
Ssn

PROJECT

Pname	<u>Pnumber</u>	Plocation
-------	----------------	-----------

S
project dept

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

5) Mapping of Binary M:N Relationship Types.

- Create new reln S to represent R.
- Include as foreign key in S, the primary keys of participating entity types.
- Include simple attributes of M:N relation type as attributes of S.

EMPLOYEE

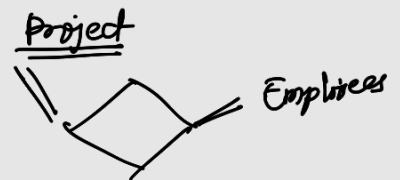
Fname	Mname	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Supr-Ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	----------	-----

Dept PK

supervisor
Ssn

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------



WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

FK

FK

6) Mapping of Multivalued Attributes

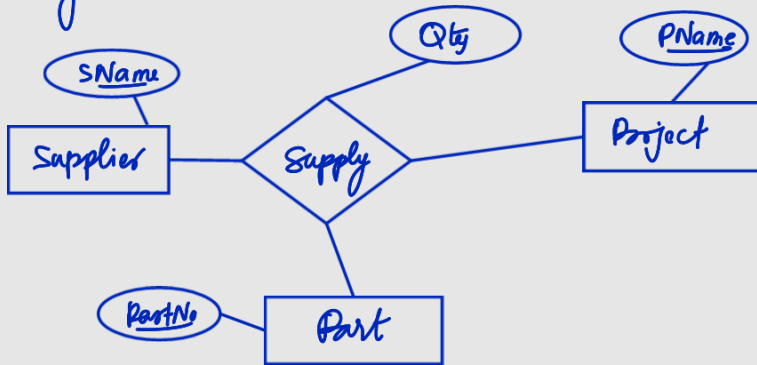
- Create a new relation R
- R will include attribute wot A + primary key attribute K as a foreign key in R.
- If multivalued attribute is composite, we include simple components.

<u>D num</u>	<u>D location</u>
--------------	-------------------

7) Mapping of N-ary relationship types

- $n > 2$, create new relation S to represent R
- Include as foreign key in S , the primary keys of participating entity types.
- Include simple attributes of the relation type as attributes of S .

eg:



SUPPLIER

SNcm | ...

PROJECT

<u>PName</u>	...
--------------	-----

PART

<u>Page No</u>	...
----------------	-----

SUPPLY

<u>SName</u>	<u>PName</u>	<u>PartNo</u>	Qty
--------------	--------------	---------------	-----